

CS-600: Software Design and Development

4-1 Assignment: Building an Efficient Data Model

Malgorzata Debska

Southern New Hampshire University

Matthew Scuteri

May 11, 2026

ACME Materials Data Model Assignment

Designing a data model is an important part of software engineering because it helps organize information in a structured and efficient way. A good data model allows applications to store, retrieve, and manage information accurately while also supporting future growth. For this assignment, a data model was created for ACME Materials to support a new application feature that stores customer and product information. The model was designed based on the provided business requirements and includes entities, attributes, relationships, keys, and constraints needed to support the application. The goal of this design is to create a system that is organized, scalable, and easy to support over time.

Entities, Attributes, and Relationships

The data model holds four main entities: **Customer**, **Customer_Address**, **Customer_Contact**, and **Product**. Each entity stands for a different type of information required by the application. The **Customer** entity stores the primary customer information. This includes a unique customer ID, customer name, and the date the customer record was created. The customer ID acts as the primary key to uniquely showing each customer in the database. The **Customer_Address** entity stores address information for customers. Since the requirements state that customers may have multiple addresses, this entity is separated from the main customer table. The table holds an address ID as the primary key and a customer ID as a foreign key that links the address back to the correct customer. Other fields include street address, city, state, ZIP code, and country. The

Customer_Contact entity stores contact information for customers. Like addresses, customers may have multiple contacts, so this information is stored in a separate table. Fields include contact ID, customer ID, first name, last name, phone number, and email address.

The **Product** entity stores product information required by ACME Materials. This includes a product ID, product name, description, quantity in stock, quantity on order, expected arrival date, weight, internal cost, and retail cost. The relationships between the entities use one-to-many cardinality. One customer can have many addresses and many contacts, while each address and contact belongs to only one customer. This relationship structure improves flexibility and reduces unnecessary duplication of data.

Constraints, Assumptions, and Considerations

Several important constraints and design considerations were used while creating this model.

Primary keys were added to each entity to ensure every record can be uniquely found. Foreign keys were used to keep relationships between related tables and enforce data integrity. Different data types were selected based on the type of information being stored. Integer data types were used for identifiers and inventory quantities because they store numeric values efficiently.

VARCHAR data types were used for names, addresses, and descriptive text because these fields may vary in length. DECIMAL data types were selected for product costs and weights to ensure correct financial and measurement calculations. DATE data types were used for expected arrival dates and record creation dates. Normalization was also considered during the design process.

Customer addresses and customer contacts were separated into different tables instead of storing all information in a single table. This helps cut duplicated data and improves long-term

maintainability. If a customer changes an address or contact, the information must be updated in one location. Another important consideration was scalability. The model was designed so future entities, such as orders, suppliers, invoices, or inventory transactions, could easily be added later without redesigning the current structure.

How the Design Supports the Application Requirements

The design directly supports the requirements provided for the ACME Materials application. The use of unique identifiers allows records to be tracked accurately and efficiently. Separating addresses and contacts into independent entities supports the requirement for multiple addresses and contacts per customer. The Product entity holds all required inventory and pricing information needed by the business. The design also supports CRUD operations because each entity can be independently created, read, updated, and dropped. This structure improves organization and allows the application to retrieve information quickly. The model follows important domain-driven design principles by organizing related business data into clear and manageable entities. According to Chandrasekaran and Krishnan (2022), well-structured domain models improve maintainability, scalability, and alignment between software systems and business needs. The use of relationships and normalized entities in this model helps create a cleaner and more efficient system design.

Strengths and Potential Limitations

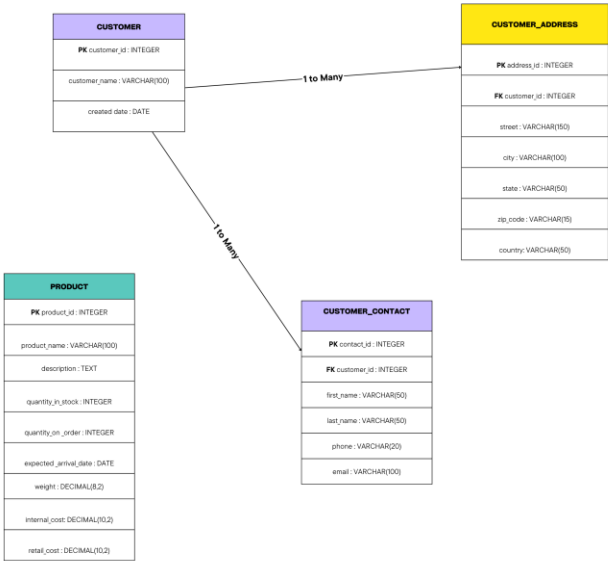
One major strength of this data model is clarity. Each entity has a purpose, making the database easier to understand and keep. The use of separate entities for customer addresses and contacts improves flexibility while reducing duplicated information. Another strength is scalability. The

design can support future expansion by allowing more entities and relationships to be added without changing the existing structure. For example, order management or supplier tracking features could be integrated into the current model with minimal disruption. The model also supports integration with other applications because relational database structures are widely supported across modern software systems and APIs. This makes it easier for the application to exchange information with reporting systems, inventory tools, or web services in the future. One possible limitation is that larger databases may eventually require indexing or optimization techniques to support high performance. As the amount of stored customer and product data increases, other database tuning may become necessary. However, the normalized structure provides a solid foundation for long-term growth and efficient data management.

In conclusion, the ACME Materials data model successfully translates business requirements into a structured relational database design. The entities, relationships, constraints, and data types support correct data storage and efficient CRUD operations while also preparing the application for future scalability and integration. Using normalization and strong relational design principles, the model provides a clear and maintainable foundation for the ACME Materials application.

ACME Materials Data Model Diagram

- LEGEND:**
- PK = Primary Key
 - A unique identifier for each record in a table.
 - FK = Foreign Key
 - A field used to connect one table to another table.
 - INTEGER = Whole number data type
 - Used for IDs and quantity values.
 - VARCHAR(n) = Variable-length text field
 - Stores text with a maximum number of characters.
 - TEXT = Long text field
 - Used for larger descriptions or notes.
 - DECIMAL(p,s) = Decimal number data type
 - Used for costs, prices, and measurements requiring precision.
 - DATE = Date value
 - Stores calendar dates.
 - 1 to Many Relationship = One record in one table can be linked to multiple records in another table.



References

Chandrasekaran, P., & Krishnan, K. (2022). *Domain-driven design with Java: A practitioner’s guide*. Packt Publishing.

